Building data course — Week 6 — RDF basics, by hand

sparql-playground / lessons

Generated for print and offline reading

Week 6 — RDF basics, by hand

Five daily lessons on Burger Barn, a made-up burger restaurant. Each day has one short script, `dayNN_rdf.py`, that reads small hand-written Turtle files next to it in `lessons/unit4-rdf-by-hand/`. Run it with `uv run python lessons/course.py run N`. Every day ends with a box on how it maps to buildings (Brick, ASHRAE 223P, Project Haystack) and a burger mini challenge; its answer is in the Try it section. Keep notes under the ignored `student-work/` folder.

Contents

- Day 26 — Your first triples: Burger Barn, written by hand
- Day 27 — Prefixes, `a` (rdf:type) and labels
- Day 28 — Literals and XSD datatypes: when “175” is not 175
- Day 29 — SPARQL: asking Burger Barn questions
- Day 30 — SPARQL checks: what’s missing?

Day 26 — Your first triples: Burger Barn, written by hand

Goal: write down facts about a (made-up) burger restaurant as **triples**, load them with rdflib, and find the typo a computer can’t see.

Read

Weeks 1–5 started from data someone else made: CSV exports and a ready-made model. This week and next you write the model yourself, by hand, about something everybody understands: a burger restaurant called **Burger Barn** (made up, like its menu).

This is how university Semantic Web courses start too: the **RDF data model** first (the W3C RDF 1.1 Primer is the standard introduction), then vocabularies on top of it. RDF stores facts as **triples**. Every triple is one short sentence with three parts:

subject (the thing)	predicate (the relationship)	object (the other thing)
Burger Barn	contains	Kitchen
Kitchen	contains	Fryer_1
Fryer_1	hasPoint	Fryer_1_OilTemp

That last row should look familiar. A restaurant contains a kitchen, a kitchen contains a fryer, and the fryer has a temperature **point**. A building contains an AHU, and the AHU has points. Same shape, smaller and tastier.

Open `day26_first_triples.ttl`. It is **Turtle**, the most readable way to write triples. One line per triple, a dot at the end, `#` for comments. Every name is a full web address (an **IRI**) in angle brackets, like `<https://example.org/burger-barn#Fryer_1>`. It doesn't have to exist on the web. It's just a name that nobody else will use by accident. (Yes, it's long. Tomorrow's prefixes fix that.)

`day26_rdf.py` does three things:

Step	What it does	rdflib
1. load	read the file into a graph	`g = Graph(); g.parse("...ttl")`
2. look	print every triple, short names only	`for s, p, o in g:` and `.fragment` (the part after `#`)
3. count	how often each predicate is used	a plain `dict`
4. ask	what does the kitchen contain?	`g.objects(kitchen, contains)`

Run

```
uv run python lessons/course.py run 26
```

```
The graph has 6 triples:
  BurgerBarn contains Kitchen
  Fryer_1 hasPoint Fryer_1_OilTemp
  Kitchen contains Fryer_1
  Kitchen contains WalkInCooler
  Kitchen contians Grill
  WalkInCooler hasPoint Cooler_AirTemp

How often each predicate is used:
  contains 3
  contians 1
  hasPoint 2

The kitchen contains: Fryer_1,
WalkInCooler
```

Count the predicates: `contains` 3 times, `hasPoint` twice, and `contians` once. That's the typo. To RDF, `contians` is a perfectly good, brand-new relationship, so there's **no error**. The grill just quietly disappears from "what does the kitchen contain?". A typo in a column name crashes your SQL; a typo in RDF makes a new word. Counting predicates (and, from Week 7, checking them against a list of known words) is how you catch it.

How this maps to buildings

> **Buildings box.** Swap the words and this file is a building model: `Building_1 contains AHU_1`, `AHU_1 hasPoint AHU_1_SAT`. **Brick** (the open building

> ontology) says exactly that with `brick:hasPoint`; **ASHRAE 223P** writes its models as the same kind of triples; and **Project Haystack** tags (`equip`, `ahu`, `equipRef`) can be exported as triples too (Week 5's `model/building_50.ttl` is one). A misspelled `brick:hasPiont` fails the same silent way, which is why real models get checked (Week 7).

Why it matters

Everything else in Weeks 6–7 is built from this one idea: small three-part facts, written by hand, that a program can read. Once facts are triples, any question about them is a pattern over triples.

Try it

1. Add a triple saying the kitchen contains an ice machine (`#IceMachine`). How many triples now?
2. Which pieces of equipment have a temperature point? Answer from the printed triples, no code.
3. **Mini challenge (Burger Barn):** fix the typo in your copy of the file. How many things does the kitchen contain now?

Answers

1. 7. One new line:

   ```
   <https://example.org/burger-
   barn#Kitchen>
   <https://example.org/burger-
   barn#contains>
   <https://example.org/burger-
   barn#IceMachine> .
   ```

2. `Fryer_1` (`Fryer_1_OilTemp`) and `WalkInCooler` (`Cooler_AirTemp`). The grill has none.
3. **3:** Fryer_1, WalkInCooler and Grill. Change `contians` to `contains` and the grill comes back. (The course's tests check this answer.)

Day 27 — Prefixes, `a` (rdf:type) and labels

Goal: write Turtle the short way with **prefixes**, say what kind of thing each thing is with `a`, and give things human-readable **labels**, then spot the missing and blank ones.

Read

Open `burger_barn.ttl`, the restaurant you'll use for the rest of the two weeks. Three new pieces of Turtle make it much shorter than yesterday's file:

Turtle	Means	Example
`@prefix bb: <https://example.org/burger-barn#> .`	`bb:` is short for that long address	`bb:Fryer_1`
`a`	short for `rdf:type`: "is a kind of"	`bb:Fryer_1 a bb:Fryer`
`;`	same subject, next predicate	`bb:Fryer_1 a bb:Fryer ; rdfs:label "Fryer 1" .`
`,`	same subject and predicate,	`bb:contains bb:Fryer_1,`

Turtle	Means	Example
	next object	`bb:Fryer_2`

`rdfs:label` is the standard predicate for a human-readable name. `bb:Fryer_1` is the **name a program uses**; `"Fryer 1"` is the **label a person reads**. Labels are values (strings), not things.

Things in a triple come in two flavours:

- a **URI** (an IRI): a thing with a name, like `bb:Fryer` or `bb:Fryer_1_OilTemp`. It can be the subject of more triples.
- a **literal**: a plain value, like `"Fryer 1"` or `175`. A literal is always an object, never a subject.

`day27_rdf.py` lists every typed thing with its label (`g.value(thing, RDFS.label)`, which returns `None` when there is none), shows Fryer_1's own triples marked URI or literal, and counts things per type.

Run

```
uv run python lessons/course.py run 27
```

```
The graph has 59 triples and 14 typed
things:
  BurgerBarn            a Restaurant
label: 'Burger Barn'
  Cooler_AirTemp        a TemperatureSensor
label: 'Walk-in cooler air temperature'
  Cooler_DoorAlarm      a DoorAlarm
label: 'Walk-in cooler door left open'
  DiningRoom            a DiningRoom
label: 'Dining room'
  Fryer_1               a Fryer
label: 'Fryer 1'
  Fryer_1_OilTemp       a TemperatureSensor
label: 'Fryer 1 oil temperature'
  Fryer_2               a Fryer
label: 'Fryer 2'
  Grill_1               a Grill
label: 'Flat-top grill'
  Grill_1_SurfaceTemp   a TemperatureSensor
label: 'Grill surface temperature'
  IceMachine_1          a IceMachine
label: None
  MainKitchen           a Kitchen
label: 'Main kitchen'
  NorthRegion           a FranchiseRegion
label: 'North region'
  SodaFountain_1        a SodaFountain
label: 'Soda fountain'
  WalkInCooler_1        a WalkInCooler
label: ''

Fryer_1's triples:
  type            Fryer
(URI)
  label           Fryer 1
(literal)
  fryTime         PT3M
```

```
(literal)
  hasPoint       Fryer_1_OilTemp
(URI)
  lastInspected  2026-10-01T09:30:00
(literal)
  oilSetpointC   175.0
(literal)
  serialNumber   BB-FRY-0001
(literal)

Things per type: {'DiningRoom': 1,
'DoorAlarm': 1, 'FranchiseRegion': 1,
'Fryer': 2, 'Grill': 1, 'IceMachine': 1,
'Kitchen': 1, 'Restaurant': 1,
'SodaFountain': 1, 'TemperatureSensor': 3,
'WalkInCooler': 1}
```

Two labels are wrong in two different ways. `IceMachine_1` has **no** label triple at all (`None`). `WalkInCooler_1` has one, but it's **blank** (`''`). A query for "things without a label" finds only the first; a person reading a dashboard gets an empty cell for the second. Day 30 finds both with SPARQL, and Day 35 writes a rule for each.

How this maps to buildings

> **Buildings box.** `ex:AHU_1 a brick:AHU ; rdfs:label "AHU 1"` is exactly how a **Brick** model types and names equipment, with the `brick:` prefix standing for `https://brickschema.org/schema/Brick#`.

> **ASHRAE 223P** uses the prefix `s223:` the same way. **Haystack** writes the type as marker tags (`equip`, `ahu`) and the label as `dis:"AHU 1"`. A missing or blank label is the classic "why does this point show up as a long ID on my graphics?" bug.

Why it matters

Types are how you ask for "every fryer" instead of naming each one, and labels are what people see. Get both into the habit of every model you write, and check for the missing and blank ones.

Try it

1. How many things are typed `bb:Fryer`? Which line of the output tells you?
2. In `burger_barn.ttl`, give the ice machine the label `"Ice machine"`. What changes in the output?
3. **Mini challenge (Burger Barn):** write a SPARQL query (or a Python loop) that lists every temperature sensor. How many does Burger Barn have?

Answers

1. 2 (`'Fryer': 2` in "Things per type").

2. Its line shows `label: 'Ice machine'`, and the graph has one more triple (60).

3. **3:** `Cooler_AirTemp`, `Fryer_1_OilTemp` and `Grill_1_SurfaceTemp`.

   ```
   for row in g.query("SELECT ?sensor
             WHERE { ?sensor a
             bb:TemperatureSensor }"):
       print(row.sensor.fragment)
   ```

   (The `bb:` prefix comes from the Turtle file, so the query doesn't need a PREFIX line.)

Day 28 — Literals and XSD datatypes: when “175” is not 175

Goal: see what type each value (literal) has, meet the common XSD datatypes, and watch a number written in quotes give a wrong answer without any error.

Read

Yesterday’s labels were **literals**: values, not things. Every literal has a **datatype**, borrowed from XML Schema (prefix `xsd:`). Turtle picks one for you when you write a bare value, and you can name one with `^^`:

You write	Datatype	Python value (rdflib)	Burger Barn example
`"degC"`	`xsd:string` (text, the default)	`str`	a sensor’s unit
`40`	`xsd:integer` (whole number)	`int`	seats
`3.0` or `"3.0"^^xsd:decimal`	`xsd:decimal` (exact decimal)	`Decimal`	cooler setpoint

You write	Datatype	Python value (rdflib)	Burger Barn example
`true`	`xsd:boolean`	`bool`	is open
`"2019-05-04"^^xsd:date`	`xsd:date` (a day)	`date`	opened on
`"2026-10-01T09:30:00"^^xsd:dateTime`	`xsd:dateTime` (day and time)	`datetime`	last inspected
`"10:30:00"^^xsd:time`	`xsd:time` (time of day)	`time`	opens at
`"PT3M"^^xsd:duration`	`xsd:duration` (how long)	`timedelta`	fry basket timer
`"fries"@en`, `"frites"@fr`	a string with a **language tag**	`str`	specialty

`PT3M` reads “a Period of Time, 3 Minutes”. `P1DT2H` would be one day and two hours.

Now the trap. Fryer_1’s setpoint is `175.0`, a number. Fryer_2’s is `"175"`, **in quotes**: text that happens to look like a number. `day28_rdf.py` prints every literal with its

datatype, then asks SPARQL "which fryers are set above 300 °C?" with `FILTER(?temp > 300)`. Neither is, so the right answer is *none*.

Run

```
uv run python lessons/course.py run 28
```

```
Literals (thing, property, text, datatype,
Python value):
  Fryer_1_OilTemp  unit           degC
string    'degC'
  BurgerBarn       seats          40
integer   40
  WalkInCooler_1   airSetpointC   3.0
decimal   Decimal('3.0')
  BurgerBarn       isOpen         true
boolean   True
  BurgerBarn       openedOn       2019-05-
04           date      datetime.date(2019,
5, 4)
  Fryer_1          lastInspected  2026-10-
01T09:30:00  dateTime
datetime.datetime(2026, 10, 1, 9, 30)
  BurgerBarn       opensAt        10:30:00
time      datetime.time(10, 30)
  Fryer_1          fryTime        PT3M
duration  datetime.timedelta(seconds=180)
  Fryer_1          oilSetpointC   175.0
decimal   Decimal('175.0')
  Fryer_2          oilSetpointC   175
string    '175'

Specialty, with language tags: [('fries',
'en'), ('frites', 'fr')]
Fryers with a setpoint above 300 C:
['Fryer_2'] <- wrong: both are set to 175
Fryers whose setpoint is text, not a
number: ['Fryer_2']
```

The query says Fryer_2 is above 300. It isn't. Comparing text with a number has no sensible answer, and rdflib quietly lets the text through. (Other SPARQL engines may

drop it instead. Either way the answer is silently wrong.) `isNumeric(?temp)` is the honest test: it finds the setpoint that is text. Day 35 adds a SHACL rule, `sh:datatype xsd:decimal`, that rejects it outright, and the ontology (Day 32) says the range of `oilSetpointC` is `xsd:decimal`.

How this maps to buildings

> **Buildings box.** BAS exports are full of numbers stored as text (“72.5”, “ON”, “1”). **Brick** stores values with explicit datatypes (a setpoint’s value is an `xsd:decimal`, a timestamp an `xsd:dateTime`); **ASHRAE 223P** does the same for its property values; **Haystack** has its own kinds (`Number` with a unit, `Bool`, `Date`, `Time`, `DateTime`) that map onto these XSD types when exported to RDF. A setpoint saved as text breaks every “above / below” fault rule, exactly like Fryer_2.

Why it matters

A literal’s datatype decides how it compares, sorts and adds up. Write numbers as numbers, dates as dates, and check the ones you didn’t write yourself.

Try it

1. Change Fryer_2's setpoint to `175.0` (no quotes) and re-run. What do the last two lines print?
2. What datatype does `"2026-10-01"` (no `^^`) have?
3. **Mini challenge (Burger Barn):** how many seconds does Fryer_1's basket timer run? Get it from Python, not by doing the math in your head.

Answers

1. Both lists are empty: nothing is above 300, and no setpoint is text.
2. `xsd:string`. Without `^^xsd:date` it is just text that looks like a date.
3. **180.** `g.value(BB.Fryer_1, BB.fryTime).toPython()` is a `timedelta`, and `.total_seconds()` gives `180.0`. (The course's tests check this answer.)

Day 29 — SPARQL: asking Burger Barn questions

Goal: write SPARQL `SELECT` queries with patterns, `OPTIONAL`, `ORDER BY` and `GROUP BY`, and read the answers in Python.

Read

SPARQL is the query language for RDF (SPARQL 1.1 Query). A query is a **pattern of triples with blanks** (variables, starting with `?`). The engine finds every way to fill the blanks so the pattern matches the graph:

```
SELECT ?thing ?label WHERE {
  bb:MainKitchen bb:contains ?thing .
# what does the kitchen contain?
  OPTIONAL { ?thing rdfs:label ?label }
# and its label, if it has one
} ORDER BY ?thing
```

SPARQL	Means	SQL you already know
`?thing`	a blank to fill	a column
a triple pattern	must match	a join condition
`OPTIONAL { ... }`	add it if it's there	`LEFT JOIN`
`ORDER BY`	sort	`ORDER BY`

SPARQL	Means	SQL you already know
`GROUP BY ?room` + `COUNT(?thing)`	count per room	`GROUP BY` + `COUNT`

`day29_rdf.py` asks three questions: what is in the kitchen (with labels), which equipment has a temperature sensor, and how many things each room contains. The `bb:` and `rdfs:` prefixes come from the Turtle file, so the queries don't need `PREFIX` lines.

Run

```
uv run python lessons/course.py run 29
```

```
In the kitchen (name, label):
   Fryer_1 'Fryer 1'
   Fryer_2 'Fryer 2'
   Grill_1 'Flat-top grill'
   IceMachine_1 None
   WalkInCooler_1 ''
Equipment with a temperature sensor:
[('Fryer_1', 'Fryer_1_OilTemp'),
('Grill_1', 'Grill_1_SurfaceTemp'),
('WalkInCooler_1', 'Cooler_AirTemp')]
Things per room: [('DiningRoom', 1),
('MainKitchen', 5)]
```

`IceMachine_1` comes back with `None`: `OPTIONAL` kept the row even though the label is missing. Without `OPTIONAL`, the ice machine would

vanish from the list, the same way an inner join drops rows.

How this maps to buildings

> **Buildings box.** "Which equipment has a temperature sensor?" is the first question every FDD tool asks a model. On a **Brick** model it is `?equip brick:hasPoint ?p . ?p a brick:Temperature_Sensor`; on **ASHRAE 223P** you follow `s223:hasProperty` from equipment to its measured properties; on **Haystack** data you ask for points with the `temp sensor` tags and an `equipRef`. Week 7 Day 35 runs today's query, word for word except the names, on a tiny Brick building.

Why it matters

SPARQL is how every question about a model gets asked, by people and by apps such as FDD rules. Once you can read a pattern, a 20-line query is just 20 small facts that must all be true at once.

Try it

1. Change the first query to drop the `OPTIONAL`. Which row disappears?

2. Count the points per piece of equipment (`bb:hasPoint`), largest first.
3. **Mini challenge (Burger Barn):** list every temperature sensor with its unit.

Answers

1. `IceMachine_1`: it has no label, so the required pattern can't match.

2. `SELECT ?equipment (COUNT(?point) AS ?n) WHERE { ?equipment bb:hasPoint ?point } GROUP BY ?equipment ORDER BY DESC(?n)` gives one point each for Fryer_1, Grill_1 and WalkInCooler_1. (The cooler's door alarm uses `bb:hasAlarm`; Day 32 shows how to count it too.)

3. Three sensors, all in `degC`:

   ```
   SELECT ?point ?unit WHERE { ?point a
   bb:TemperatureSensor ; bb:unit ?unit }
   ```

   (The course's tests check this answer.)

Day 30 — SPARQL checks: what's missing?

Goal: use `FILTER NOT EXISTS`, `!BOUND` and `STRLEN` to find missing points, missing labels and blank labels.

Read

The most useful questions about a model are often about what is **not** there. SPARQL has two ways to ask:

Question	SPARQL	How it works
fryers with no temperature point	`FILTER NOT EXISTS { ?fryer bb:hasPoint ?p . ?p a bb:TemperatureSensor }`	keep the row only if the inner pattern finds nothing
equipment with no label	`OPTIONAL { ?e rdfs:label ?l } FILTER(!BOUND(?l))`	the optional part left `?l` empty
equipment with a blank label	`?e rdfs:label ?l . FILTER(STRLEN(?l) = 0)`	the label exists but has zero characters

`day30_rdf.py` runs all three.

Run

```
uv run python lessons/course.py run 30
```

```
Fryers with no temperature point:
['Fryer_2']
Kitchen equipment with no label at all:
['IceMachine_1']
Kitchen equipment with a blank label:
['WalkInCooler_1']
```

Fryer_2, the ice machine and the walk-in cooler: three different mistakes, three different queries. This is **closed-world** checking: "if it isn't written down, it's missing". That sounds obvious, but Week 7 shows that OWL reasons the other way (open world), and Day 35 turns these queries into SHACL rules, the standard way to write closed-world checks.

How this maps to buildings

> **Buildings box.** "VAVs without a zone temperature sensor", "points with no unit", "equipment with no label": these are the first checks anyone runs on a new **Brick** or **Haystack** model, and they are the same `FILTER NOT EXISTS`

> queries. **ASHRAE 223P** and Brick ship these kinds of rules as SHACL shapes, so a tool can check a model before an FDD rule trusts it.

Why it matters

An FDD rule that silently skips the AHU with no supply-air temperature looks like "no faults". A check like today's turns that silence into a to-do list.

Week 6 further reading

- W3C, RDF 1.1 Primer and RDF 1.1 Turtle
- W3C, SPARQL 1.1 Query Language (sections 2–5 and 8 cover this week)
- W3C, XML Schema datatypes (just the table of built-in types)
- rdflib documentation

Try it

1. Give Fryer_2 a temperature point in `burger_barn.ttl` (a new `bb:Fryer_2_OilTemp a bb:TemperatureSensor`). Which list empties?
2. Why doesn't `FILTER(!BOUND(?l))` find the walk-in cooler?
3. **Mini challenge (Burger Barn):** which points have no `bb:unit`? (Look at points

linked with `bb:hasPoint` or `bb:hasAlarm`.)

Answers

1. “Fryers with no temperature point” becomes `[]`.

2. The cooler *has* a label triple, so `?l` is bound (to `""`). Only `STRLEN` sees that it’s empty.

3. **`Cooler_DoorAlarm`**, which is fair: a door alarm is open or closed and has no unit.

   ```
   SELECT ?point WHERE {
     ?equipment bb:hasPoint|bb:hasAlarm ?
   point .
     FILTER NOT EXISTS { ?point bb:unit ?
   unit }
   }
   ```

   (The course’s tests check this answer.)

Further reading — Week 6

The lessons first, then these for exact behavior.

- W3C: RDF 1.1 Primer
- W3C: RDF 1.1 Turtle
- W3C: SPARQL 1.1 Query Language
- W3C: XML Schema datatypes
- rdflib documentation
- Brick: relationships